

Agents and Tool Use

ReAct · Tool Schemas · Agent Memory · Error Handling · Text-to-SQL Agent · Evaluation

WEEK 3 · DAY 19

Friday, July 24 2026

Day 19 of 84

Sec	Topic	Coverage
Sec 1	Tool Schemas	JSON schema per tool: name, description, parameters
Sec 2	ReAct Pattern	Reason+Act+Observe loop; ReActAgent with MAX_STEPS=10
Sec 3	Tool Executors	SQLAgentToolkit: execute_sql, retrieve_schema, get_sample_rows
Sec 4	Agent Memory	Working memory (session) + long-term memory (persisted SQLs)
Sec 5	Agent System Prompt	5-step reasoning protocol + rules + dialect + memory ctx
Sec 6	Error Handling	AgentErrorType enum; SQL error, timeout, MAX_STEPS, refusal
Sec 7	Full Agent (TextToSQLAgent)	Integration: D17 retriever + D18 prompts + D19 loop
Sec 8	Agent Evaluation	answered>95%, correct_kw>85%, avg_steps<4, latency<8s
Sec 9	FAANG Patterns	Google parallel / Microsoft planner / Amazon / Meta / Netflix
Sec 10	Architect's Lens	Tool granularity, step budget, memory, error recovery
Sec 11	Exercises + Summary	ReAct trace, error injection, long-term memory test

TEXT-TO-SQL AGENT COMPLETE (Days 13-19): D13 foundation + D14 vLLM/FastAPI + D15 RAG schema + D16 indexing + D17 hybrid retrieval + D18 generation (CoT+faithfulness) + D19 agent loop (ReAct+tools+memory). SLOs: answered>95%, correct_kw>85%, avg_steps<4, latency<8s.

Section 1 — Tool Schemas

Tools are described as JSON schemas passed to the LLM. The model reads the description and parameters to decide when and how to call each tool.

```
TOOLS = [
  {
    "type": "function",
    "function": {
      "name": "execute_sql",
      "description": "Execute a SELECT SQL query. Never use for writes.",
      "parameters": {
        "type": "object",
        "properties": {
          "sql": {"type": "string", "description": "The SELECT SQL to execute."},
          "explain": {"type": "boolean", "description": "Run EXPLAIN instead (syntax check).", "default": False},
        },
        "required": ["sql"],
      },
    },
  },
  {
    "type": "function",
    "function": {
      "name": "retrieve_schema",
      "description": "Search vector DB for table schemas relevant to a question.",
      "parameters": {
        "type": "object",
        "properties": {
          "question": {"type": "string", "description": "NL question to search schema for."},
          "schema_name": {"type": "string", "description": "Database schema to search."},
          "top_k": {"type": "integer", "description": "Tables to retrieve (default 5).", "default": 5},
        },
        "required": ["question", "schema_name"],
      },
    },
  },
  {
    "type": "function",
    "function": {
      "name": "get_sample_rows",
      "description": "Fetch sample rows to understand data format. Use when column values unclear.",
      "parameters": {
        "type": "object",
        "properties": {
          "table_name": {"type": "string"},
          "schema_name": {"type": "string"},
          "limit": {"type": "integer", "default": 3},
        },
        "required": ["table_name", "schema_name"],
      },
    },
  },
  {
    "type": "function",
    "function": {
```

```

        "name": "clarify_question",
        "description": "Ask user for clarification when request is genuinely ambiguous.",
        "parameters": {
            "type": "object",
            "properties": {
                "question": {"type": "string", "description": "The clarifying question to ask."},
            },
            "required": ["question"],
        },
    },
},
]

```

Section 2 — The ReAct Pattern

ReAct = Reasoning + Acting. The model alternates between thinking and tool calling. Each observation feeds into the next reasoning step.

```

# ReAct trace: "How many users signed up in the last 7 days?"
# Turn 1: THOUGHT → retrieve schema | OBSERVE: users table DDL
# Turn 2: THOUGHT → validate SQL | OBSERVE: EXPLAIN plan (syntax OK)
# Turn 3: THOUGHT → execute SQL | OBSERVE: [{"count": 847}]
# Turn 4: FINAL ANSWER: "847 users signed up in the last 7 days."

```

```

class ReActAgent:
    """
    Terminates when model produces a message WITHOUT tool_calls (final answer).
    MAX_STEPS = 10 prevents infinite loops.
    """
    MAX_STEPS = 10

    async def run(self, user_question: str, system_prompt: str,
                  tenant_id: str) -> dict:
        messages = [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_question},
        ]
        steps = []
        async with httpx.AsyncClient(base_url=self.url, timeout=60) as client:
            for step in range(self.MAX_STEPS):
                r = await client.post("/v1/chat/completions", json={
                    "model": self.model,
                    "messages": messages,
                    "tools": self.tools,
                    "tool_choice": "auto", # model decides when to stop calling tools
                    "temperature": 0.0,
                    "max_tokens": 512,
                })
                msg = r.json()["choices"][0]["message"]
                if not msg.get("tool_calls"):
                    return {"answer": msg["content"], "steps": steps, "n_steps": step+1}

                messages.append(msg)
                for tc in msg["tool_calls"]:
                    fn_name = tc["function"]["name"]
                    fn_args = json.loads(tc["function"]["arguments"])

```

```
        result = await self._execute_tool(fn_name, fn_args, tenant_id)
        steps.append({"step": step+1, "tool": fn_name, "args": fn_args, "result": result})
        messages.append({"role": "tool", "tool_call_id": tc["id"],
                        "content": json.dumps(result)})

    return {"answer": None, "error": f"Exceeded {self.MAX_STEPS} steps", "steps": steps}
```



```

@dataclass
class AgentWorkingMemory:
    conversation_id: str
    tenant_id: str
    schema_name: str
    retrieved_schemas: dict[str, str] = field(default_factory=dict)
    executed_sqls: list[dict] = field(default_factory=list)

    def remember_schema(self, table_name: str, ddl: str):
        """Cache retrieved schema – avoid re-fetching the same table."""
        self.retrieved_schemas[table_name] = ddl

    def already_retrieved(self, table_name: str) -> bool:
        return table_name in self.retrieved_schemas

    def context_summary(self) -> str:
        """Injected into next system prompt to prevent duplicate retrievals."""
        lines = []
        if self.retrieved_schemas:
            lines.append("ALREADY RETRIEVED (do not retrieve again):")
            for tbl in self.retrieved_schemas: lines.append(f" - {tbl}")
        return "\n".join(lines) if lines else ""

# ■■ LONG-TERM: Persisted memory (cross-session, per tenant/user) ■■■■■■■■■■
class AgentLongTermMemory:
    async def save_successful_sql(self, tenant_id, user_id, question, sql):
        """Store validated question→SQL pairs. Surfaces as few-shot examples."""
        conn = await asyncpg.connect(self.db_url)
        await conn.execute("""
            INSERT INTO agent_memory (tenant_id, user_id, question, sql, created_at)
            VALUES ($1, $2, $3, $4, NOW())
            ON CONFLICT (tenant_id, user_id, question) DO UPDATE SET sql=$3
        """, tenant_id, user_id, question, sql)
        await conn.close()

    async def find_similar_past_sql(self, tenant_id, question, top_k=3) -> list[dict]:
        """Return recent successful SQLs for the same tenant (few-shot examples)."""
        conn = await asyncpg.connect(self.db_url)
        rows = await conn.fetch("""
            SELECT question, sql FROM agent_memory
            WHERE tenant_id=$1 ORDER BY created_at DESC LIMIT 50
        """, tenant_id)
        await conn.close()
        return [{"question": r["question"], "sql": r["sql"]} for r in rows[:top_k]]

    async def save_user_preference(self, tenant_id, user_id, key, value):
        """Persist preferences: date_format, schema_alias, default_filter, etc."""
        conn = await asyncpg.connect(self.db_url)
        await conn.execute("""
            INSERT INTO agent_preferences (tenant_id, user_id, key, value)
            VALUES ($1, $2, $3, $4) ON CONFLICT (tenant_id, user_id, key) DO UPDATE SET value=$4
        """, tenant_id, user_id, key, value)
        await conn.close()

```

Section 5 — Agent System Prompt

```
AGENT_SYSTEM_PROMPT = """You are a Text-to-SQL agent for the {schema_name} database.
Tools: retrieve_schema, execute_sql, get_sample_rows, clarify_question.
```

```
REASONING PROTOCOL (follow in order):
```

1. RETRIEVE: Call retrieve_schema first. Never guess table names.
2. INSPECT: Call get_sample_rows if column values or formats are unclear.
3. VALIDATE: Call execute_sql with explain=True to check SQL syntax.
4. EXECUTE: Call execute_sql to get results.
5. ANSWER: Plain-English answer citing the SQL used.

```
RULES:
```

- Only reference tables retrieved in this session. Never hallucinate tables.
- Qualify all column names: table.column (users.email, not email).
- Only SELECT queries. execute_sql rejects writes automatically.
- Use clarify_question only when question is genuinely ambiguous.
- Do not re-retrieve a table already in ALREADY RETRIEVED list.

```
DATABASE: {dialect} | SCHEMA: {schema_name} | TENANT: {tenant_id}
```

```
{working_memory_context}"""
```

```
# Example injected working_memory_context after step 1:
```

```
# "ALREADY RETRIEVED (do not retrieve again):
```

```
#   - users
```

```
#   - orders
```

```
# LAST SQL EXECUTED: SELECT COUNT(users.id) FROM users WHERE..."
```

```
def build_agent_system_prompt(schema_name, dialect, tenant_id, memory=None) -> str:
    ctx = memory.context_summary() if memory else ""
    return AGENT_SYSTEM_PROMPT.format(
        schema_name=schema_name, dialect=dialect,
        tenant_id=tenant_id, working_memory_context=ctx)
```

Section 6 — Error Handling

```
from enum import Enum
```

```
class AgentErrorType(Enum):
```

```
    SQL_SYNTAX_ERROR    = "sql_syntax_error"    # SQL failed to parse or execute
    SCHEMA_NOT_FOUND    = "schema_not_found"    # retrieval returned empty
    TOOL_TIMEOUT        = "tool_timeout"        # tool exceeded 10s limit
    MAX_STEPS_EXCEEDED  = "max_steps_exceeded"  # hit MAX_STEPS without answer
    LLM_REFUSAL         = "llm_refusal"         # model refused the request
    PARSE_ERROR         = "parse_error"         # could not parse tool args
```

```
class RobustReActAgent(ReActAgent):
```

```
    TOOL_TIMEOUT_SECS = 10
```

```
    async def _execute_tool_safe(self, fn_name, fn_args, tenant_id) -> dict:
```

```
        executor = self.executors.get(fn_name)
```

```
        if not executor:
```

```
            return {"error": f"Unknown tool: {fn_name}",
                    "error_type": AgentErrorType.PARSE_ERROR.value}
```

```
        try:
```

```

        result = await asyncio.wait_for(
            executor(**fn_args, tenant_id=tenant_id),
            timeout=self.TOOL_TIMEOUT_SECS)
    if "error" in result and "sql" in fn_args:
        result["error_type"] = AgentErrorType.SQL_SYNTAX_ERROR.value
        result["suggestion"] = (
            "SQL had an error. Check column names against schema. "
            "Re-retrieve schema if needed, then fix and retry.")
    return result
except asyncio.TimeoutError:
    return {"error":f"Tool {fn_name} timed out after {self.TOOL_TIMEOUT_SECS}s",
            "error_type":AgentErrorType.TOOL_TIMEOUT.value,
            "suggestion":"Use cached schema if available and proceed."}
except Exception as e:
    return {"error":str(e),"error_type":AgentErrorType.PARSE_ERROR.value}

# ERROR RECOVERY PRIORITY:
# 1. SQL syntax error → inject error message; agent self-corrects (1 retry typical)
# 2. Schema not found → increase top_k 5→10, retry retrieve_schema
# 3. Empty result → call get_sample_rows to verify data exists, then re-reason
# 4. MAX_STEPS hit → return partial result + last SQL attempted + error message

```


Section 7 — Complete TextToSQLAgent

Full integration: HybridSchemaRetriever (Day 17) + SQLPromptBuilder (Day 18) + ReAct loop (Day 19). Long-term memory surfaces past SQLs as few-shot examples.

```
class TextToSQLAgent:
    async def answer(self, question: str, schema_name: str,
                     tenant_id: str, user_id: str) -> dict:
        # 1. Working memory for this session
        memory = AgentWorkingMemory(
            conversation_id=f"{tenant_id}-{user_id}-{int(datetime.utcnow().timestamp())}",
            tenant_id=tenant_id, schema_name=schema_name)

        # 2. Long-term memory: past successful SQLs → few-shot context
        past_sqls = await self.lt_memory.find_similar_past_sql(tenant_id, question)

        # 3. Build system prompt (working memory + past examples)
        system_prompt = build_agent_system_prompt(schema_name, self.dialect, tenant_id, memory)
        if past_sqls:
            system_prompt += ("\n\nSIMILAR PAST QUERIES:\n" +
                              "\n".join(f"Q: {p['question']}\nSQL: {p['sql']}" for p in past_sqls))

        # 4. Run ReAct loop
        result = await self.agent.run(question, system_prompt, tenant_id)

        # 5. Persist successful SQL to long-term memory
        if result.get("answer") and not result.get("error"):
            sql_steps = [s for s in result.get("steps", [])
                          if s["tool"]=="execute_sql" and "error" not in s.get("result", {})]
            if sql_steps:
                await self.lt_memory.save_successful_sql(
                    tenant_id, user_id, question, sql_steps[-1]["args"]["sql"])
        return result

# Example: "Which product category had highest revenue last quarter?"
# Step 1: retrieve_schema("product category revenue", "ecommerce")
#         → products, orders, order_items tables retrieved
# Step 2: execute_sql(explain=True) → syntax OK
# Step 3: execute_sql(full) → [{"category": "Electronics", "revenue": 2400000}]
# Answer: "Electronics had highest revenue last quarter at $2.4M."
# Steps: 3 | Latency: ~2.9s
```

Section 8 — Agent Evaluation

```
class AgentEvaluator:
    async def evaluate(self, cases: list[AgentEvalCase], tenant_id="eval") -> AgentEvalResult:
        res = AgentEvalResult(total=len(cases))
        for case in cases:
            t0 = time.perf_counter()
            result = await self.agent.answer(case.question, case.schema_name, tenant_id, "eval")
            latency = (time.perf_counter() - t0) * 1000
            res.avg_latency_ms += latency
            res.avg_steps += result.get("n_steps", 0)
```

```

if result.get("answer") and not result.get("error"):
    res.answered += 1
    ans = result["answer"].lower()
    if all(kw.lower() in ans for kw in case.expected_answer_kw):
        res.correct_kw += 1

if result.get("n_steps",99) <= case.max_steps:
    res.within_step_budget += 1

if case.gold_sql:
    sql_steps = [s for s in result.get("steps",[])]
                if s["tool"]=="execute_sql" and not s["args"].get("explain")]
    if sql_steps:
        gen = " ".join(sql_steps[-1]["args"]["sql"].lower().split())
        gold = " ".join(case.gold_sql.lower().split())
        if gen == gold: res.sql_matches_gold += 1

n=len(cases); res.avg_steps/=n; res.avg_latency_ms/=n
return res

# PRODUCTION TARGETS:
# answered / total          > 0.95  (agent produces answer almost always)
# correct_kw / total        > 0.85  (answer contains expected information)
# within_step_budget / total> 0.90  (stays under step budget per query)
# avg_steps                 < 4.0   (efficient; most tasks in 3-4 steps)
# avg_latency_ms            < 8000  (< 8s end-to-end including all tool calls)

```

Section 9 — FAANG Agent Patterns

Google — Parallel Tool Execution (Gemini Function Calling)

Gemini models support parallel function calling: multiple tool calls in one response, executed concurrently. In Google's internal data analysis agents, schema retrieval + sample row fetch dispatch simultaneously — cutting avg step count from 4.2 to 2.8. Implementation: parallel tool_calls share the same tool_call_id namespace; results batched into one tool message. End-to-end latency reduction: 35-40% on multi-table queries.

Microsoft — Semantic Kernel Planner (Azure AI Studio)

Planner model (GPT-4o-mini) first decomposes the user question into a confirmed sequence of tool calls. Executor model (GPT-4o) then runs only confirmed steps. Planning and execution are separated: the expensive model never wastes tokens on deliberation. Result: 28% reduction in total token usage on multi-step enterprise Q&A; tasks. Called the Stepwise Planner in Semantic Kernel's open-source framework.

Amazon — Bedrock Agents with Knowledge Base Integration

Bedrock Agents integrates RAG retrieval as a built-in tool backed by Amazon OpenSearch. Enforced first-call structure: Knowledge Base retrieval is always the first tool call. This grounds the agent in retrieved context before any action. Amazon's internal data: 71% reduction in hallucinated tool calls vs free-form agent ordering.

Meta — Verified ReAct (LLaMA Agents, Research Tooling)

After each tool result, a lightweight verifier (fine-tuned DeBERTa, 125M params) checks consistency between the tool result and the stated reasoning step. If verification score < 0.7, agent is forced to re-reason before proceeding. This 'verified ReAct' prevents bad early steps from silently corrupting later steps. Result: 15% reduction in final answer errors on multi-step queries vs standard ReAct.

Netflix — Human-in-the-Loop at clarify_question (Content Decision Support)

When an agent calls `clarify_question`, the query pauses and surfaces in Slack. A data analyst responds in Slack; the response injects back into the agent loop. 3-5% of queries pause for human clarification vs forcing the agent to guess. Result: answer correctness improved from 79% to 91% on ambiguous content questions. Design insight: let the agent know when it doesn't know.

Section 10 — Architect's Lens

TOOL GRANULARITY:

Too coarse: one "database_tool" that does everything → agent can't express needs
 Too fine: separate tools for table names vs columns → wastes steps on orchestration
 RIGHT: one tool per coherent operation (4 tools for Text-to-SQL is near-optimal)

STEP BUDGET (expected steps per query complexity):

Simple (single table COUNT): 2 steps (retrieve + execute)
 Medium (2-table JOIN): 3 steps (retrieve + validate + execute)
 Hard (subquery + sample needed): 4-5 steps
 MAX_STEPS=10 is a safety ceiling, not a target.
 ALERT when avg_steps > 5 – agent is confused or schema is too complex.

MEMORY DESIGN:

Working memory: message history + schema cache → prevents re-retrieval within session
 Long-term memory: past validated question→SQL pairs → dynamic few-shot at next session
 Rule: NEVER persist raw LLM output. ONLY persist user-validated SQL + question pairs.

ERROR RECOVERY ORDER:

1. SQL syntax error → inject specific error; agent self-corrects (1 retry)
2. Schema not found → increase top_k from 5 to 10; retry retrieve_schema
3. Empty result → get_sample_rows to verify data exists; re-reason
4. MAX_STEPS hit → return partial result with last SQL + error explanation

PARALLEL TOOL CALLS:

When to use: multiple INDEPENDENT retrievals (e.g., retrieve 3 tables simultaneously)
 Gain: 35-40% latency reduction on multi-table queries
 Cost: more complex message assembly; some models don't support it yet
 Support: Gemini 1.5+, GPT-4o (OpenAI), Claude 3+ (function calling)

LATENCY BUDGET (3-step query on production):

Schema retrieval (Qdrant hybrid): ~250ms
 SQL execution (small table): ~50ms
 LLM call per step (8B on GPU): ~800ms
 Typical 3-step query: ~3.3s (within 8s SLO)
 With parallel tool calls: ~2.2s (35% reduction)

Section 11 — Exercises and Summary

Exercise 1: ReAct Loop Trace

```
# Run agent on 3 questions of increasing complexity:
# 1. "Count all users" → expect 2 steps
# 2. "Total revenue by product category" → expect 3-4 steps
# 3. "Users who ordered more than average spend" → expect 4-5 steps
# For each: print step trace (tool, args, result snippet)
# Verify: does agent ALWAYS retrieve schema before querying? If not – bug.
```

Exercise 2: Schema Error Injection

```
# Pass a wrong schema_name to retrieve_schema deliberately
# Observe: how does agent handle empty retrieval result?
# GOOD: agent calls clarify_question OR reports "schema not found"
# BAD: agent hallucinates table names → fix with rule 1 in system prompt
```

Exercise 3: Long-Term Memory Test

```
# Run same question twice for same tenant/user
# First run: agent uses 3-4 steps (no prior memory)
# Second run: check if past SQL surfaces as few-shot example in system prompt
# Measure: does step count drop on second run?
# Expected: second run uses 2 steps (validate + execute with cached pattern)
```

Summary

Concept	Key Point
Agent = loop	Input → LLM → tool call → observation → LLM → ... → final answer
Tool schemas	JSON schema: name, description, parameters, required fields
ReAct pattern	Reason+Act+Observe; terminates on message without tool_calls
MAX_STEPS=10	Hard cap prevents infinite loops; avg target < 4 steps
SQLAgentToolkit	execute_sql (safety gate), retrieve_schema, get_sample_rows, clarify_question
Safety gate	Only SELECT/WITH/EXPLAIN — rejects writes in execute_sql
Working memory	Session schema cache + SQL log; prevents duplicate retrieval
Long-term memory	Persisted question→SQL pairs; surfaces as few-shot next session
Agent system prompt	5-step protocol (retrieve→inspect→validate→execute→answer)
Error handling	SQL error→self-correct; timeout→fallback; MAX_STEPS→partial
TextToSQLAgent	Full integration: D17 retriever + D18 prompts + D19 agent loop
SLOs	answered>95%, correct_kw>85%, avg_steps<4, latency<8s

NEXT: Day 20 — Multi-Agent Systems

Orchestrator-worker architecture, agent communication protocols, parallel agent execution, specialized sub-agents (retrieval/generation/validation), and building a multi-agent pipeline for the Text-to-SQL system.